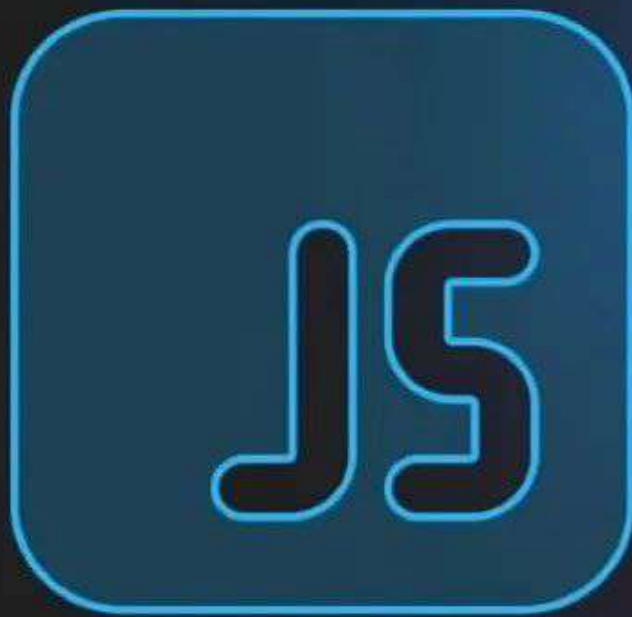


DOM

Manipulation



DOM Manipulation

For DOM manipulation, we use JavaScript to access, modify, or create these elements, making our web pages come alive without the need to reload them.

In this post we are going to look at:

- Create Elements
- Insert an elements
- Modifying Contents
- Replacing Elements
- Remove Elements
- Updating Element Attributes

Create Element

Imagine you want to add a new element dynamically to your page. The `createElement` method comes to the rescue!

With `textContent` and `innerHTML`, we can set or retrieve Text and HTML content, including tags and nested elements

```
const newItem = document.createElement('div');

// Set contents with textContents
newItem.textContent = 'New Item';

// OR Set contents with innerHTML
innerHTML = '<h2>New Item</h2>'
```

Insert Elements

After successfully creating element, now we want to **insert it on DOM**.

We have two nifty techniques: **appendChild** to add an element as the **last child** of a parent and **insertBefore** to inserts an element **before an existing child**.

Example

```
<div id="parent">  
  <p>Coffee</p>  
  <p>Tea</p>  
</div>
```

```
▼ <div id="parent">  
  <p>Coffee</p>  
  <p>Tea</p>  
</div>
```

1. appendChild()

```
const createEl = document.createElement('p')  
createEl.innerHTML = "water"  
const parentDiv = document.getElementById("parent")  
parentDiv.appendChild(createEl)
```

```
▼ <div id="parent">  
  <p>Coffee</p>  
  <p>Tea</p>  
  <p>water</p>  
</div>
```

2. insertBefore()

```
const createEl = document.createElement('p')  
createEl.innerHTML = "water"  
const parentDiv = document.getElementById("parent")  
const referenceItem = parentDiv.firstElementChild;  
parentDiv.insertBefore(createEl, referenceItem)
```

```
▼ <div id="parent">  
  <p>water</p>  
  <p>Coffee</p>  
  <p>Tea</p>  
</div>
```


Example

```
<div id="parent">
  <p>Coffee</p>
  <p>Tea</p>
</div>
```

```
▼ <div id="parent">
  <p>Coffee</p>
  <p>Tea</p>
</div>
```

1. appendChild()

```
const createEl = document.createElement('p')
createEl.innerHTML = "water"
const parentDiv = document.getElementById("parent")
parentDiv.appendChild(createEl)
```

```
▼ <div id="parent">
  <p>Coffee</p>
  <p>Tea</p>
  <p>water</p>
</div>
```

2. insertBefore()

```
const createEl = document.createElement('p')
createEl.innerHTML = "water"
const parentDiv = document.getElementById("parent")
const referenceItem = parentDiv.firstChild;
parentDiv.insertBefore(createEl, referenceItem)
```

```
▼ <div id="parent">
  <p>water</p>
  <p>Coffee</p>
  <p>Tea</p>
</div>
```

Replacing Elements

The ability to **replace elements** in the DOM adds a new dimension to our **dynamic content** updates.

Say hello to **replaceChild**, which lets us swap existing elements with fresh ones.

```
<div id="myDiv">This is a div element.</div>

const divElement = document.getElementById('myDiv');

const newParagraph = document.createElement('p');
newParagraph.textContent = 'This is a new paragraph';

divElement.parentNode.replaceChild(newParagraph, divElement);
```

Removing Elements

To keep our dynamic content in check, we need to know **how to remove elements**.

That's where the **removeChild** method swoops in!

It allows us to **bid farewell** to specific elements or items **based on user actions** or conditions.

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<button id="removeItemBtn">Remove Last Item</button>

// JS
const listElement = document.getElementById('myList');
const removeItemBtn = document.getElementById('removeItemBtn');

removeItemBtn.addEventListener('click', () => {
  const lastItem = listElement.lastChild;
  listElement.removeChild(lastItem);
});
```


Conclusion

DOM manipulation empowers developers to **create dynamic** and interactive web experiences.

Mastering methods like **inserting, removing, replacing elements**, updating attributes enables seamless user interactions.

As always, I **hope you enjoyed** the post and **learned something new**.

If you have **any queries** then **let me know** in the comment box.